Chapter 1 — About this Guide

1.1 Scope and Philosophy

This guide teaches you to build, evaluate, ship and operate LLM-powered systems. It is organised around one belief: **AI engineering is software engineering with a probabilistic component**. The model is the least reliable part of your system, so most of the craft is in the scaffolding around it: validation, retrieval, evaluation, observability, and cost control.

Principles used throughout:

1. **Measure before you optimise.** No prompt, chunk size, or model swap ships without an eval number.
2. **Simplest thing that works.** A prompt beats RAG beats fine-tuning beats pre-training, in cost and in speed to iterate. Move down the list only when evals prove you must.
3. **Failure is the default.** Every network call, parse, and model output can fail; design the unhappy path first.
4. **Code over slides.** Every concept has a runnable snippet.

1.2 Prerequisites

Area	Needed level
Programming	Comfortable in Python: functions, classes, packaging, virtualenvs
Math	Vectors, dot products, matrix multiplication, basic probability, derivatives
Web	HTTP verbs, JSON, what a REST API is
Tools	Git, a terminal, Docker basics

If you lack the math, Chapter 5 re-derives what is needed. If you lack Python depth, Chapter 3 is mandatory.

1.3 Reading Strategy

- **Fast path (application engineer, 2 weeks):** 2 → 4 → 6 → 7 → 9 → 10 → 14 → 17.
- **Full path (AI engineer, 12 weeks):** in order, doing the labs in Chapter 18.
- **Platform/infra path:** 3 → 4 → 5 → 12 → 13 → 14 → 15.
- **Interview prep:** 16, 19, then skim sections' *Anti-Patterns*, which map to system-design questions.

Each concept section has four parts: **Conceptual Deep Dive**, **Cross-Connection Scenarios**, **Production Anti-Patterns**, **Minimal Working Code**. Read anti-patterns even if you skip the rest.

1.4 Repository Layout and Conventions

```
chapters/   Markdown source (01..19)
pdfs/       generated PDFs
scripts/    build_pdfs.py, audit.py, required_concepts.json
```

Conventions: Python 3.10+, type hints everywhere, `pydantic` v2 for schemas, `httpx` for HTTP, `asyncio` for concurrency. SDK model names change often; treat any literal model string as a placeholder and read the

vendor docs.

Conceptual Deep Dive

The guide is layered: foundations (2–5), interaction with models (6–7), knowledge (8–9), quality (10), autonomy (11), customisation (12), operations (13–15), people and practice (16–19). Each layer depends on the ones before it.

Cross-Connection Scenarios

A customer-support bot touches nearly every chapter: prompts (6), structured output (7), RAG (9), eval (10), guardrails (14), serving (13), CI gates (15). Use Chapter 17 to see the assembled picture.

Production Anti-Patterns

- Reading the guide linearly without building anything.
- Copying snippets to production without the error handling chapters (4, 14).
- Treating benchmark leaderboards as evidence about *your* task.

Minimal Working Code

A project bootstrap check that verifies your environment:

```
import importlib
import sys

REQUIRED = ["httpx", "pydantic", "numpy"]

def check() -> list[str]:
    missing = []
    for pkg in REQUIRED:
        try:
            importlib.import_module(pkg)
        except ImportError:
            missing.append(pkg)
    if sys.version_info < (3, 10):
        missing.append("python>=3.10")
    return missing

if __name__ == "__main__":
    problems = check()
    print("OK" if not problems else f"Missing: {problems}")
```

1.5 How to Verify Claims in a Fast-Moving Field

Conceptual Deep Dive

Model names, prices, context limits, SDK signatures and benchmark rankings change monthly; principles (sampling, attention, retrieval, evaluation, reliability) change slowly. Sort every fact you read into one of three bins: **durable** (math, protocols, algorithms), **semi-durable** (framework architecture, index types), and **perishable** (model ids, prices, leaderboards). Verify perishable facts against primary sources: vendor docs, release notes, the model card, the GitHub repo's changelog, and the paper. Prefer reproducing a claim on your own data over trusting a blog benchmark. Edge cases: documentation that lags the SDK, deprecated parameters that silently no-op, and "preview" features that change shape before general availability.

Cross-Connection Scenarios

Chapter 10 provides the eval harness to test claims on your data; Chapter 15 pins versions so your results stay reproducible; Chapter 16 covers communicating uncertainty about perishable facts to stakeholders.

Production Anti-Patterns

- Hard-coding a model id and price in ten places.
- Copying a snippet from a year-old tutorial that uses a removed API.
- Trusting an unattributed benchmark table.

Minimal Working Code

```
from dataclasses import dataclass
from datetime import date

@dataclass
class Fact:
    claim: str
    source: str
    verified_on: date
    perishable: bool = True

    def stale(self, today: date, max_age_days: int = 90) -> bool:
        return self.perishable and (today - self.verified_on).days > max_age_days

facts = [Fact("Model X context = 200k", "vendor docs", date(2026, 1, 5)),
         Fact("Softmax is shift-invariant", "math", date(2020, 1, 1), perishable=Fals
e)]
print([f.claim for f in facts if f.stale(date(2026, 10, 22))])
```

```
import json
import pathlib

def load_model_config(path: str = "models.json") -> dict:
    """Single source of truth for perishable facts; code never hard-codes ids/prices."
;""
    p = pathlib.Path(path)
    if not p.exists():
        return {"default": {"id": "REPLACE_ME", "usd_per_mtok_in"
: 0.0, "usd_per_mtok_out": 0.0}}
    return json.loads(p.read_text(encoding="utf-8"))
```

1.6 Setting Up a Reproducible Lab Environment

Conceptual Deep Dive

Create one virtual environment per project, pin dependencies in a lockfile, keep API keys in environment variables loaded from an untracked .env, and write a smoke test that proves the environment works before you start a lab. For GPU labs decide early: local GPU, rented cloud GPU (Chapter 13), or CPU-only with small models. Budget guardrails matter on day one: set provider spend limits and billing alerts before running loops that call paid APIs. Edge cases: Windows path and line-ending differences, CUDA/driver/PyTorch version mismatches, and corporate proxies that break TLS.

Cross-Connection Scenarios

Python packaging (3.6), secrets (14.5), cost tracking (2.4), containers (13.3).

Production Anti-Patterns

- Committing `.env` files.
- Global `pip install` into the system interpreter.
- Running unbounded loops against a paid API with no spend limit.

Minimal Working Code

```
import os
import sys

def env_report() -> dict:
    rep = {"python": sys.version.split()[0], "api_key_set": bool(os.getenv("LLM_
API_KEY"))}
    try:
        import torch
        rep["cuda"] = torch.cuda.is_available()
    except ImportError:
        rep["cuda"] = None
    return rep

print(env_report())
```

```
import time

class SpendGuard:
    """Hard stop for lab scripts that call paid APIs."""
    def __init__(self, max_usd: float):
        self.max, self.spent, self.t0 = max_usd, 0.0, time.time()

    def charge(self, usd: float) -> None:
        self.spent += usd
        if self.spent > self.max:
            raise RuntimeError(f"spend limit ${self.max} exceeded (${self.spent:.2f})")

guard = SpendGuard(5.0)
guard.charge(0.02)
```

1.7 Chapter Dependency Map and Role-Based Tracks

Conceptual Deep Dive

Chapters form a directed graph, not a line. Foundations (2, 3, 4, 5) feed interaction (6, 7), which feeds knowledge (8, 9) and agents (11); evaluation (10) cuts across everything and should start as early as Chapter 6; operations (13, 14, 15) wrap whatever you built; people chapters (16, 17, 18) apply throughout. Role tracks: **application engineer** (2, 4, 6, 7, 9, 10, 14, 17), **retrieval/search engineer** (3, 5, 8, 9, 10, 13), **ML platform engineer** (3, 4, 5, 12, 13, 14, 15), **evaluation/safety engineer** (6, 10, 14, 15), **agent engineer** (6, 7, 11, 10, 14). Hard prerequisites: tokenisation (5.3) before chunking (9.2); structured output (7) before agents (11); evaluation basics (10.1) before prompt optimisation (6.6) or fine-tuning (12). Edge cases: readers with ML backgrounds can skip 5 but should read 4 and 14; readers new to software should do 3 and 4 labs before anything else.

Cross-Connection Scenarios

Reading strategy (1.3), learning roadmap phases (18), interview preparation (16.3).

Production Anti-Patterns

- Starting with agents before you can evaluate a single prompt.
- Skipping reliability chapters because the demo worked.
- Learning serving internals before shipping a simple API integration.

Minimal Working Code

```
DEPENDS = {
    "rag": {"embeddings", "chunking", "evaluation"},
    "embeddings": {"tokenization", "python_async"},
    "agents": {"structured_output", "tool_use", "evaluation"},
    "structured_output": {"prompting", "pydantic"},
    "fine_tuning": {"evaluation", "dataset_curation", "gpu_memory"
```

```
;},
    "evaluation": {"python_async", "statistics"},
}

def study_order(goal: str) -> list[str]:
    order, seen = [], set()
    def visit(n):
        if n in seen:
            return
        seen.add(n)
        for d in sorted(DEPENDS.get(n, ())):
            visit(d)
        order.append(n)
    visit(goal)
    return order

print(study_order("agents"))
```

```
def missing_prereqs(goal: str, known: set[str]) -> list[str]:
    needed = set(study_order(goal)) - {goal}
    return sorted(needed - known)

print(missing_prereqs("fine_tuning", {"statistics", "python_async"}))
```

1.8 Responsible AI Principles for Practitioners

Conceptual Deep Dive

Responsible practice is engineering, not slogans. Principles with concrete controls: **fairness** (slice evals by demographic and language groups; compare error rates, not just averages), **transparency** (tell users when they interact with AI, show sources, document limits), **accountability** (named owners, audit logs, escalation paths), **privacy** (minimisation, retention limits, deletion), **safety** (red-teaming, guardrails, kill switches), **human oversight** (approval for high-impact decisions), and **reliability** (abstention over confident error). Higher-risk domains (health, finance, hiring, education, legal, children) need additional review, narrower scope and human-in-the-loop by default. Edge cases: models that perform worse for dialects or non-English users, automation bias (users over-trusting fluent outputs), and feedback loops where model outputs become training data.

Cross-Connection Scenarios

Governance and incident response (14.7), evaluation slices (10.1), communication of limitations (16.2).

Production Anti-Patterns

- A single aggregate accuracy figure hiding poor performance for a subgroup.
- Shipping high-stakes decisions with no human override path.
- Training on model outputs without provenance tracking.

Minimal Working Code

```
def slice_gap(results: list[dict], key: str, metric: str = "corre
ct") -> dict:
    groups: dict[str, list[float]] = {}
    for r in results:
        groups.setdefault(r[key], []).append(float(r[metric]))
    means = {g: sum(v) / len(v) for g, v in groups.items()}
    return {"means": means, "max_gap": max(means.values()) - min(means.values())}

data = [{"lang": "en", "correct": 1}] * 90 + [{"lang": "en&q
uot;, "correct": 0}] * 10 \
    + [{"lang": "sw", "correct": 1}] * 6 + [{"lang": "sw&qu
ot;, "correct": 0}] * 4
```

```
print(slice_gap(data, "lang"))
```

```
RISK_TIERS = {
    "minimal": {"human_review": False, "disclose_ai": False},
    "limited": {"human_review": False, "disclose_ai": True},
    "high": {"human_review": True, "disclose_ai": True},
}

def controls_for(use_case: str, domain: str) -> dict:
    high_domains = {"health", "finance", "hiring", "legal", "
;education", "children"}
    tier = "high" if domain in high_domains else ("limited" if use_case == "chat
" else "minimal")
    return {"tier": tier, **RISK_TIERS[tier]}

print(controls_for("chat", "hiring"))
```